

# Compiling a Hopfield Network into Clockless Logic

Aarav Vidhawan · Advised by Prof. Bill Lin

Department of Electrical and Computer Engineering, UC San Diego

Summer Research Internship Program (SRIP), 2026 · [github.com/a-vidhawan/ECESRIP](https://github.com/a-vidhawan/ECESRIP)

---

## Summary

---

A Hopfield associative memory updates each neuron as  $s_i = \text{sign}(\sum_j W_{ij} s_j)$ . Normally that becomes a multiply-accumulate and a comparator. This project asked what happens if you compile each neuron straight into **two-level Boolean logic** instead, and then remove the clock so the network settles as a combinational feedback circuit.

Both halves work. Both have a catch that only shows up when you measure.

**Synthesis.** An associative memory doesn't need to be correct everywhere — only on the states it actually visits. Specifying each neuron over a bounded operating region and leaving the rest as don't-cares shrinks the specification from  $2^d$  rows to  $M \cdot \sum_{k \leq h} C(d, k)$ , which is polynomial in fan-in where the full table is exponential. Measured: **31–54 product terms vs 627–2,918** at fan-in 16, and still feasible at fan-in 32, where the complete table has  $4.29 \times 10^9$  rows and can't be written down at all.

**Timing.** Update order can come from propagation delay instead of a clock edge. Colour the coupling graph, give each colour class a different delay, and coupled neurons can never commit at the same instant. Two conditions on that turned out to matter, and neither was obvious up front: the constraint is on the delay **values**, not the colour labels, and the delay elements have to be **inertial** rather than transport.

A full network was built at  $N = 256$  and 12,357 product terms, and checked in RTL against its own behavioural model on 240 of 240 vectors.

---

## 1. The problem

---

A Hopfield network is a recurrent network of binary threshold units with a symmetric weight matrix that defines an energy function. Stored patterns sit at local minima. Recall means starting near a pattern and letting the network descend into it. That's attractive for hardware because it's a *settling* process, not a computation — no instruction stream, and in principle no clock.

Two things get in the way.

**The truth table is exponential.** A neuron with fan-in  $d$  has a  $2^d$ -row table. At  $d = 32$  that's  $4.29 \times 10^9$  rows — not just slow to minimise, impossible to write down. Existing LUT-based neural network work (NullaNet, LogicNets, PolyLUT) gets around this by sampling which inputs a trained network actually produces, which gives a frequency rather than a guarantee.

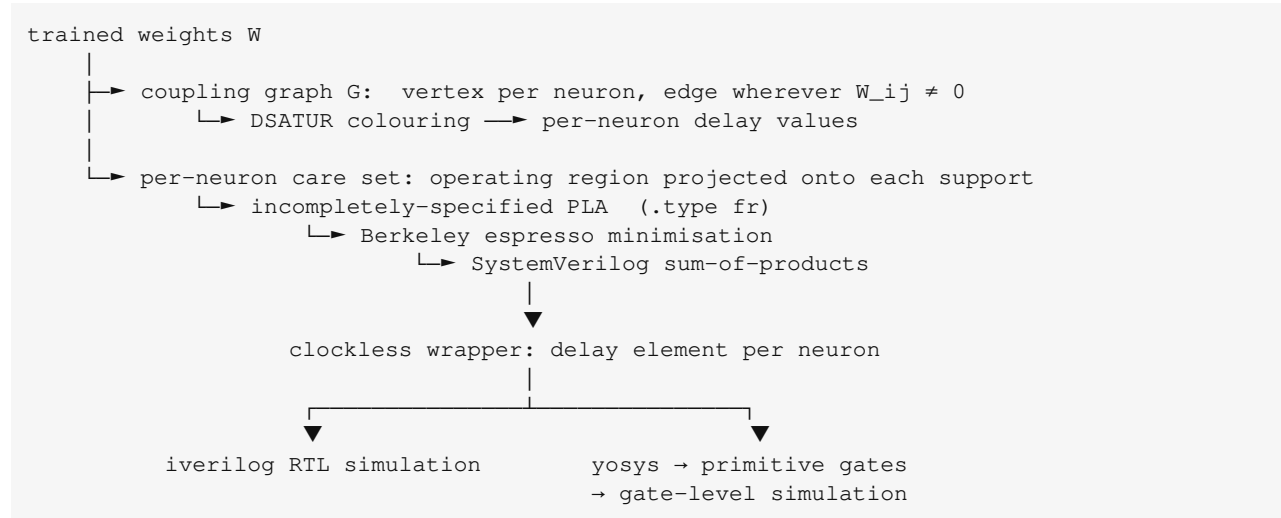
**Dropping the clock breaks convergence.** Hopfield convergence proofs assume serialised updates. If two coupled neurons update at once, each acts on the other's stale value and the network can cycle instead of settling. Measured exhaustively at  $N = 16$ : **60.2% of all  $2^{16}$  states cycle** under fully synchronous updating. Something has to impose an order, and there's no clock to do it.

---

## 2. Approach

---

The pipeline turns a trained weight matrix into synthesizable RTL:



Two decisions do most of the work.

**The care set is derived, not sampled.** The operating region is every state within Hamming distance  $h$  of a stored pattern. Its projection onto neuron  $i$ 's support is enumerated directly — the projection of a corrupted pattern is just that pattern's projection with at most  $h$  of *those* bits flipped — which gives a closed-form size of  $M \cdot \sum_{k \leq h} C(d, k)$ . That's a guarantee over a stated radius, not an observed frequency.

**Order comes from delay.** Each neuron's output passes through a delay element in its feedback path. Different colour classes get different delays, so no two coupled neurons commit together. No clock generator, no distribution network, no sequencer.

---

## 3. Results

---

### 3.1 Don't-care synthesis

| fan-in | full table rows | care rows | care % | terms (don't-care) | terms (full)           |
|--------|-----------------|-----------|--------|--------------------|------------------------|
| 16     | 65,536          | 2,768     | 4.22%  | 31–54 (mean 40)    | 627–2,918 (mean 1,385) |
| 24     | 16,777,216      | 9,300     | 0.055% | 10–58 (mean 27)    | can't enumerate        |
| 32     | 4,294,967,296   | 21,956    | 0.001% | 5–27 (mean 14)     | can't enumerate        |

The saving is in what has to be **specified**; the term count follows from that. At fan-in 48 espresso itself times out at 30 minutes, so the ceiling is minimiser runtime rather than term count.

**The cost.** Outside the operating region the function is unspecified, so espresso picks freely and the minimised network drifts away from the exact threshold network. Agreement on uniformly random states is **2.5%**. Inside the operating region they're indistinguishable, including at Hamming distance 5 — past the radius-3 care set they were built from. Fine for recall, not for adversarial input.

Because the network is recurrent this needed checking rather than assuming. An unspecified input that

later comes up can create a fixed point the target function doesn't have, which has no equivalent in a feed-forward network.

### 3.2 The constraint is on delay values

Partitioning by parity of neuron index — the obvious first try — fails. On an  $N = 16$  network with 43 coupling edges at 35.8% density the chromatic number is **6**, and parity leaves **19 of 43 coupled pairs (44.2%)** inside a shared class. The highest-degree neuron shares a class with six of its eleven neighbours.

The  $2 \times 2$  that settled it:

|                       | distinct delay values | identical delay values |
|-----------------------|-----------------------|------------------------|
| proper colouring      | 100% settled          | 0% settled             |
| parity (no colouring) | 0% settled            | —                      |

The top-right cell is the result. A valid proper colouring with all classes assigned the same delay meets every graph-theoretic requirement and settles in none of the trials. So the constraint is on delay values, not colour labels — a distinction that doesn't exist in a clocked design, where a colour simply is a phase. The project's own verifier had this bug and would have passed the 0% schedule.

**The particular values don't matter.** Five delay families — consecutive multiples, powers of two, prime multiples, coprime offsets, and an irrational ratio — all settle from 100% of random starts at  $N = 16, 32$  and  $64$ . Twelve random permutations of the same six primes over the same six classes give a standard deviation of 0.00. Distinctness is the whole requirement.

### 3.3 The delay element has to be inertial

This one came out of an anomaly: a zero-delay *reference* model was settling less often than the glitchy gate-level designs it was supposed to be the reference for, which can't happen if the schedule works.

An **inertial** delay cancels a pending transition if its cause goes away before the delay elapses. A **transport** delay queues every transition and delivers all of them. Since each neuron evaluates continuously rather than once per pass, a neuron's target can revert after a transition is scheduled and before it commits. A transport element then writes that superseded value onto neighbours that have already moved.

Same networks, same colourings, same delay values, same initial states — only the delay semantics differ:

| N   | inertial | transport |
|-----|----------|-----------|
| 16  | 100%     | 0.7%      |
| 32  | 100%     | 15.7%     |
| 64  | 100%     | 14.7%     |
| 128 | 100%     | 7.0%      |

Commensurate delay values realigning was ruled out first, which left the semantics as the only explanation.

This isn't a mitigation you can add or leave off — it's what makes the ordering work without a periodic timing reference. A clocked design gets the same protection a different way, by sampling only after the logic has settled. A delay-sequenced design has no sampling instants, so it has to come from the delay element itself, and a plain delay line gives none of it.

### 3.4 End-to-end at N = 256

N = 256, M = 4, fan-in 16, care radius 3,  $\chi = 4$ , no delay-value conflicts. 12,357 product terms over 256 neurons, emitted as SystemVerilog and run in Icarus Verilog.

| HD | n  | RTL settled        | RTL recall         | RTL = simulator |
|----|----|--------------------|--------------------|-----------------|
| 0  | 60 | 100.0 [94.0–100.0] | 100.0 [94.0–100.0] | 100.0           |
| 1  | 60 | 100.0 [94.0–100.0] | 100.0 [94.0–100.0] | 100.0           |
| 3  | 60 | 100.0 [94.0–100.0] | 100.0 [94.0–100.0] | 100.0           |
| 5  | 60 | 100.0 [94.0–100.0] | 100.0 [94.0–100.0] | 100.0           |

240 of 240. The last column is what justifies using the faster event-driven simulator for larger networks — at N = 256, not at N = 4096.

### 3.5 Capacity, variation, optimisation

**Capacity.** Margin-based retraining (Krauth–Mertens minover under a sparsity mask, with  $\kappa$  chosen as the largest feasible value) stores every pattern with  $\geq 95\%$  recall at a loading of  $\alpha = M/N = 0.5$ , against **0.138** for the classical outer-product rule. Corruption tolerance grows with size:  $\geq 90\%$  recall out to 5% of bits flipped at N = 64, 16% at N = 128, **19% at N = 256**.

**Process variation.** Randomly perturbing each neuron's delay leaves settling at 100% out to  $\pm 348\%$  spread at  $3\sigma$ . More usefully, applying variation to a *degenerate* schedule — all classes on the same nominal delay — lifts settling from 67% to 100%, because continuous variation makes equal delays distinct with probability one. Silicon variation helps here. The thing to avoid is delays made equal *by construction*, which is what identical buffer chains would give.

**Optimisation.** Mapping MAX-CUT onto the same machine with  $w = -A$ , the network settles on 100% of instances and reaches **99.5%** of the best cut found by any method tested, at roughly **100×** lower simulated cost than single-flip simulated annealing at equal restarts. A fixed point is a locally optimal cut; there's no asymptotic claim here.

## 4. What didn't work

Every headline number is regenerated from source by `audit_claims.py`. Final tally: **14 verified, 2 revised, 2 retracted**.

**Retracted.** (i) "32 universal oscillators never converge under any configuration" — false. Every graph-coloured schedule settles all 32 across 18 independent schemes; the original claim generalised from having only tried parity variants. (ii) "Noise is a third delay mode" — the noise RTL emitted byte-identical delays to depth mode, because `round(d + U(-0.5, 0.5))` almost always returns `d`. Every three-way mode comparison was really two-way.

**Revised.** An early estimate put the LUT approach at  $2.4\text{--}2.8\times$  smaller than threshold gates. Synthesis says  **$1.52\times$  smaller on an ASIC proxy and  $1.19\times$  larger on FPGA** — 4-bit weights are enough where the estimate assumed 8, and 6-LUT packing favours the adder tree. Area is not the selling point.

**A baseline that wins.** At N = 64, M = 4 a nearest-match CAM is **2,858 gates to our 7,020**, and recalls 100% at every Hamming distance where this design manages  $\sim 57\%$ . The verdict flips with care radius — at radius 2 we're  $2.1\times$  smaller — but the loading sweep is clear: 4/4 patterns stored at M = 4, 6/8 at M

= 8, 0/16 at  $M = 16$ . This design runs out of storage before it runs out of area.

**Where the failures actually are.** Decomposing recall failures from random starts: spurious convergence is  $\sim 75\%$ , oscillation  $\sim 2\%$ . Scheduling fixes the oscillation column. Only loading fixes the other one.

---

## 5. Relation to prior work

---

The scheduling mechanism is not new, and working that out took a fair amount of the project.

Colour-partitioned update ordering for recurrent stochastic networks goes back to **Geman & Geman (1984)**. **Gonzalez et al. (AISTATS 2011)** built the Chromatic Gibbs sampler on the same energy argument. In hardware, **Aadit et al. (Nature Electronics, 2022)** describe their FPGA Ising machine as "*a low level hardware-level implementation of chromatic Gibbs sampling*", and their bibliography collects more colour-block hardware: an FPGA parallel Gibbs accelerator (FlexGibbs, FCCM 2019), FPGA Ising annealing processors (Yoshimura et al., 2016, 2017), and an FPGA restricted Boltzmann machine (Patel et al., 2020).

That same paper also states the sizing constraint — "*the MAC must finish its computation before the next color block is updated*" — and reports that deliberately violating it (*overclocking*, related to Hogwild!-Gibbs) improves time-to-solution.

Two things came through a full-text review of the closest references. The combination of colour-partitioned ordering *and* no periodic timing reference didn't turn up anywhere. Neither did the inertial requirement in §3.3. The operating-region don't-care derivation is also untouched by that work, which derives don't-cares by sampling activations rather than in closed form from a bounded region.

---

## 6. Method

---

Results carry an evidence tier: **T1** measured in RTL, **T2** measured by a tool (yosys, espresso), **T3** from the event-driven simulator, **T4** analytical estimate.

Proportions are Wilson 95% confidence intervals rather than the normal approximation, because a lot of these estimates sit at exactly 0 or 1, where the normal interval collapses to zero width.

`audit_claims.py` regenerates every headline number from the source data, which is how the two retractions in §4 were caught.

---

## 7. Limitations

---

- **The delays are simulation constructs.** Every delay here is a Verilog `#` directive, which synthesis tools strip. Real hardware would need physical delay elements — buffer chains, current-starved inverters, programmable delay lines — and none were built or synthesized. Only the combinational neuron logic went through yosys; the scheduling wrapper did not.
- **No silicon, no PDK timing.** Area figures are yosys cell counts after technology-independent mapping.
- **RTL verification stops at  $N = 256$ .** Larger numbers come from the event-driven simulator, validated against RTL at  $N = 16$  and  $N = 256$  and nowhere beyond.
- **Random bipolar patterns only.** No correlated or real data, which would change the basin

geometry.

- **Off-region behaviour is unspecified by construction** — 2.5% agreement with the exact network on uniformly random states.
- **The dual-rail comparison is incomplete.** Inertial delay was re-measured on a corrected testbench; the C-element variant was not, and its numbers come from a testbench with a known premature-readout bug.

---

## 8. Reproducing

---

Needs `python3` (`numpy`, `matplotlib`), `iverilog`, `yosys`, and Berkeley `espresso` on `PATH`.

```
python3 phase2/paper/audit_claims.py          # regenerate every headline number
python3 phase2/clockless/rtl_n256.py --N 256 --M 4 --degree 16 --radius 3
python3 phase2/clockless/analyze_coupling.py   # colouring vs parity
python3 phase2/phase10_glitch/inertial_required.py # inertial vs transport
```

---

## References

---

1. J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *PNAS* 79(8):2554–2558, 1982.
2. S. Geman and D. Geman. Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. *IEEE TPAMI PAMI*-6(6):721–741, 1984.
3. W. Krauth and M. Mézard. Learning algorithms with optimal stability in neural networks. *J. Phys. A* 20(11):L745, 1987.
4. J. Gonzalez, Y. Low, A. Gretton, C. Guestrin. Parallel Gibbs sampling: from colored fields to thin junction trees. *AISTATS*, PMLR 15:324–332, 2011.
5. N. A. Aadit et al. Massively parallel probabilistic computing with sparse Ising machines. *Nature Electronics* 5:460–468, 2022.
6. S. Nikhar, S. Kannan, N. A. Aadit, S. Chowdhury, K. Y. Camsari. All-to-all reconfigurability with sparse and higher-order Ising machines. *Nature Communications* 15:8977, 2024.
7. G. G. Ko, Y. Chai, R. A. Rutenbar, D. Brooks, G.-Y. Wei. FlexGibbs: reconfigurable parallel Gibbs sampling accelerator for structured graphs. *FCCM*, 2019.
8. Y. Umuroglu, Y. Akhauri, N. J. Fraser, M. Blott. LogicNets: co-designed neural networks and circuits for extreme-throughput applications. *FPL*, 2020.
9. S. H. Unger. *Asynchronous Sequential Switching Circuits*. Wiley-Interscience, 1969.
10. R. K. Brayton, G. D. Hachtel, C. McMullen, A. Sangiovanni-Vincentelli. *Logic Minimization Algorithms for VLSI Synthesis*. Kluwer, 1984.